

What the Serving Engine Does with the Physics

Inference Fundamentals, Part 2: continuous batching, the paged KV cache, and every optimization classified by what it attacks

Part 1B established that an LLM request has two phases with different limits: prefill is bound by arithmetic, decode is bound by memory bandwidth, and batching is nearly free until compute catches up or memory runs out. This part is about the software that turns a model file into a service, and what it does with those facts. By the end you should be able to hear the name of any inference optimization and say which of three things it attacks, and therefore when it will help and when it will do nothing.

Where does the time go? For more than a decade that was my question, asked of large data platforms on behalf of enterprises: was a query reading, computing, or waiting for a slot; was the compute sized for the concurrency the system actually saw or the concurrency someone guessed; what deserved a cache; and what did all of it cost per query, in terms finance could act on. More recently the question has moved to language models: agentic systems, and the open-source and frontier models beneath them. I came to inference expecting a new set of rules. What I found was the same discipline in different units: two hardware limits, a handful of formulas connecting them to what a user feels, and the familiar problem of which lever a given symptom is asking for. This series is for readers with that background who want to understand inference well enough to run it.

A model file on disk is not a service. Between the file and the user sits the serving engine: a program that loads the weights onto the GPU, listens for requests over HTTP, decides which requests run in which step, manages the KV cache, and streams tokens back. The well-known engines are vLLM, SGLang and TensorRT-LLM. Most of the performance a user experiences is decided by how the engine schedules work, so the first half of this article is about four of its mechanisms. The second half is the catalog of optimizations, organized so that you can reason about them rather than memorize them.

Continuous batching

The obvious way to batch is static: collect N requests, run them together until the longest one finishes, then collect the next N. The trouble is that answers vary wildly in length. If one request in the batch generates 800 tokens and the others generate 50, the other slots sit idle for 750 steps, and any new request that arrives waits for the whole batch to drain. GPU time is wasted and latency is terrible.

Static batching vs continuous batching

Static: wait for the whole batch to finish



Continuous: refill each slot the moment it frees, every step



Time runs left to right. Each box is one request occupying a batch slot for its decode steps.

Continuous batching, sometimes called iteration-level scheduling, makes the batch a thing that changes at every step. When a request finishes, its slot is released immediately and a waiting request takes it on the very next step. A new arrival waits at most one decode step, a few milliseconds, to join. This is the single biggest reason modern engines reach high utilization, and it is what a vendor means by "token-level batching that lets requests join an active batch mid-stream."

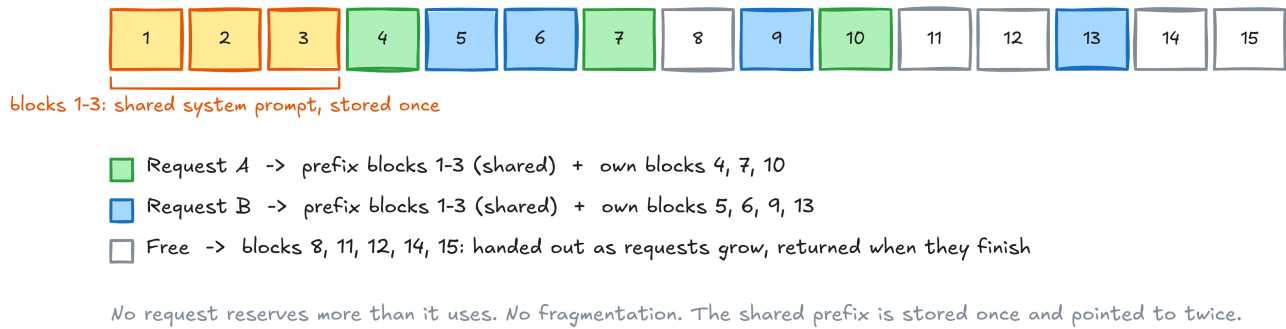
It is worth pausing on why this is possible at all. In Part 1B, decode was memory-bound: the weights are read once per step and applied to every request in the batch. That means requests in a batch do not need to be at the same stage. One can be on its 5th token and another on its 700th; each just needs one more forward pass, and the weights serve them all. Continuous batching is the scheduling consequence of the physics.

The paged KV cache

Early engines reserved KV cache memory per request for the maximum possible context length, up front, in one contiguous block. A request that used 500 tokens of a 4,096-token reservation wasted 88 percent of its allocation, and the GPU "ran out of memory" while most of it sat empty. Freed blocks of different sizes could not be reused efficiently, so memory fragmented.

vLLM's PagedAttention borrowed the operating system's idea of virtual memory. The KV cache is carved into fixed-size blocks, 16 tokens each by default in vLLM. A request is given blocks one at a time as it grows and returns them when it finishes. There is no reservation and no fragmentation, and the number of requests that fit in the same memory roughly doubles or triples.

KV cache memory as 16-token blocks (PagedAttention)



Every serious engine now does some version of this. It also enables something that becomes important below: if two requests begin with the same 2,000-token system prompt, they can point to the same blocks instead of each holding a copy. That sharing is the foundation of prefix caching.

Scheduling and preemption

At every step the scheduler makes three decisions. Which waiting requests to admit, which means running their prefill. Which running requests to advance by one decode step. And what to do if the KV cache blocks run out.

That last case is preemption: the engine evicts a running request's KV cache to make room, and either recomputes it later (redoing its prefill) or swaps it out to CPU memory and back. Preemption is what Part 1B's memory cliff looks like from inside the engine. The evicted request's latency jumps by a full prefill, and overall throughput drops because work is being redone. Engines count preemptions, and a rising count is the clearest sign that a deployment's concurrency setting is too high for the context lengths it is actually seeing.

Streaming

Tokens are sent to the client as soon as they are produced, usually over server-sent events or a WebSocket, rather than collected and returned as one response. Streaming does not make the model any faster. What it changes is which metric the user feels. A non-streaming caller waits the full end-to-end time and sees nothing until then. A streaming user waits only the time to first token, then reads along at the inter-token pace. That is why every chat product streams, why time to first token is the chat metric, and why an API used by a program (which cannot read along) cares about end-to-end time instead.

The optimizations, classified

Every technique in this field attacks one of three things:

1. **Bytes read per step.** Decode is bound by these. Anything that shrinks the weights or the KV cache, or spreads them across more GPUs, speeds up decode.
2. **Arithmetic per token.** Prefill is bound by this. Anything that skips work or does it with fewer operations speeds up prefill and time to first token.
3. **Interference in the schedule.** A long prefill stalls everyone's decode on the same GPU. Anything that separates or interleaves them protects latency under mixed load.

Knowing which one a technique attacks tells you when it will help and when it will not. Here is the catalog.

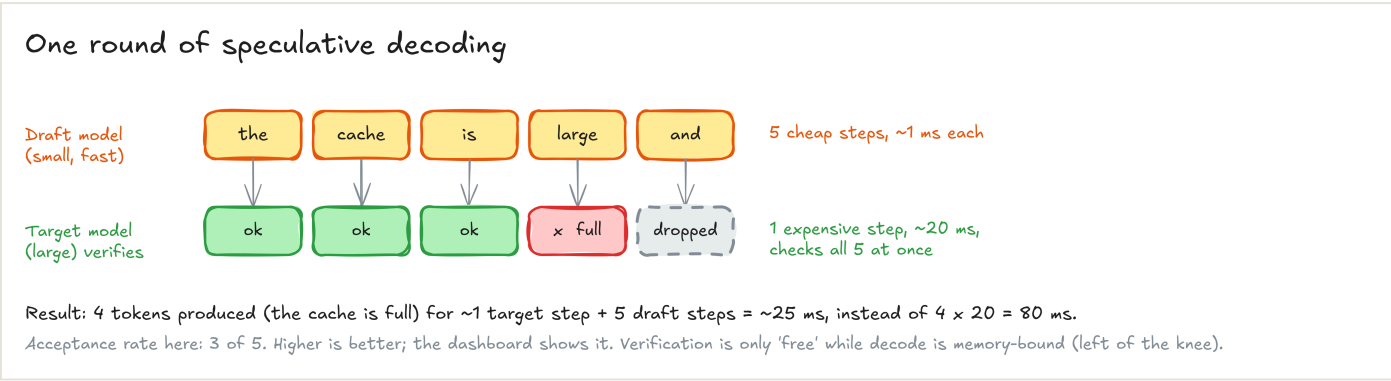
TECHNIQUE	ATTACKS	HOW IT WORKS	HELPS MOST WHEN	HURTS OR DOES NOTHING WHEN
Quantization (FP8, FP4, INT8, INT4)	Bytes per step; memory footprint	Store weights, and optionally the KV cache, in fewer bits. H100 does FP8 math natively; B200 adds FP4. Applied after training.	Decode-heavy workloads; models that barely fit; any case where bandwidth is the limit	Quality-sensitive tasks without an evaluation; aggressive 4-bit on small models; prefill-dominated workloads gain less
Tensor parallelism	Bytes per step per GPU; memory footprint	Each layer's weight matrices are split across GPUs; the GPUs exchange partial results at every layer over NVLink.	Large models; lowering inter-token latency; making room for KV cache	Small models, where the exchange costs more than it saves; across servers without a fast interconnect cache
Expert parallelism	Memory footprint of mixture-of-experts models	Different experts live on different GPUs. DeepSeek V3: 671B parameters total, 37B active per token, so compute is like a 37B model and memory like a 671B one.	MoE models at scale	Dense models (does not apply)
Speculative decoding	Sequential steps per output token	A small draft proposes several tokens; the large target verifies them all in one forward pass. Explained below.	Low batch with spare compute; predictable text such as code, JSON, translation	High batch, where verification is no longer free; a badly matched draft with a low acceptance rate
Prefix caching	Prefill compute; TTFT	Reuse the KV blocks of a shared prompt prefix and run prefill only on the new part. Explained below.	Agents and RAG with large shared prompts; multi-turn chat	Prompts that never repeat; very short prompts; routing that scatters repeats across replicas
Chunked prefill	Scheduling interference	A long prompt's prefill is split into chunks interleaved with other requests' decode steps.	Mixed workloads with a long-input tail	Pure batch, where there is nobody to protect; it slightly lengthens the long request's own TTFT
Prefill prioritization	TTFT under load	The scheduler admits waiting requests' prefill ahead of running requests' decode.	Chat, where TTFT matters more than ITL	Voice, where ITL is the constraint and you would prioritize decode instead
Disaggregated prefill/decode	Scheduling interference, at the pool level	Separate GPU pools run prefill and decode; the KV cache is shipped between them. Explained below.	Large scale with long inputs and long outputs	Small deployments, where the transfer and the second pool cost more than they save
KV cache offload	Memory ceiling	Cold KV blocks move to CPU RAM, local SSD or network storage and are pulled back on demand.	Long contexts, many idle sessions, a prefix cache larger than GPU memory	Hot data needed every step, where offload adds latency
Custom and fused kernels	Arithmetic efficiency and overhead	Several GPU operations are combined into one so intermediate results never go to memory; attention kernels tuned for specific context lengths. TensorRT-LLM compiles these ahead of time.	Everything; this is where TensorRT-LLM's edge comes from	Costs compile time and flexibility; brand-new architectures lag
Structured outputs	Correctness, not performance	A JSON schema is compiled to a state machine; at each step, tokens that would violate it are masked before sampling.	Tool calling and extraction	Not a performance lever, but know it so you never propose "retry until the JSON is valid"
Smaller model, distillation, routing	Everything, by doing less work	Serve a small model for the easy majority and route hard cases to the large one.	Cost pressure; high volume; tasks with a clear easy/hard split	Tasks where quality is the product; without an evaluation to measure the regression

Three of these come up constantly and deserve to be understood step by step.

Speculative decoding, step by step

Decode is slow because it is sequential: one token per step, and each step reads all the weights. Speculative decoding attacks the sequential part.

A small, fast draft model guesses the next several tokens, perhaps five, one after another. Because it is small, each of its steps is cheap. Then the large target model checks all five guesses in a single forward pass. This is the trick: checking five tokens costs the target about the same as producing one, because decode is memory-bound, and reading the weights once serves five positions just as it served one. Tokens the target agrees with are accepted; at the first disagreement the target's own choice is used and the rest are discarded. The output is exactly what the target would have produced on its own. Only the number of expensive steps changed.



Now the limits, which are the part worth being able to explain. The trick depends on the target's verification being nearly free, and that is only true while decode is memory-bound, that is, left of Part 1BBB's knee. At high batch the GPU is compute-bound, verifying five tokens costs five tokens' worth of arithmetic, and the draft model is also consuming GPU time, so the technique slows things down. Good runtimes switch it off dynamically at high batch for exactly this reason.

It also depends on the draft guessing well. A draft that is accepted 20 percent of the time wastes most of its work. The ideal pairing is a large model and its own small sibling (Llama 70B with Llama 8B as draft), and the workloads where it shines are ones with predictable text: code, JSON, translation. Variants like Medusa and EAGLE attach small prediction heads to the target model instead of running a separate draft; lookahead decoding generates and verifies candidate n-grams in parallel with no draft model at all. The acceptance rate is the health metric, and a serving platform should expose it.

Prefix caching, step by step

Prefill is expensive and time to first token depends on it. But a great deal of prefill work is repeated. Every request to a chat product begins with the same system prompt. Every call in an agent loop carries the same tool definitions. Every turn of a conversation re-sends the whole history. In each case the first thousands of tokens are identical to tokens the engine has already processed, and their keys and values are already sitting in someone's KV cache.

Prefix caching: skip the prefill you already did



Two conditions: the prefix must match exactly from the first token (put the stable part first, the variable part last), and the request must land on the replica that holds it (KV cache-aware routing). The KV cache hit rate is the proof.

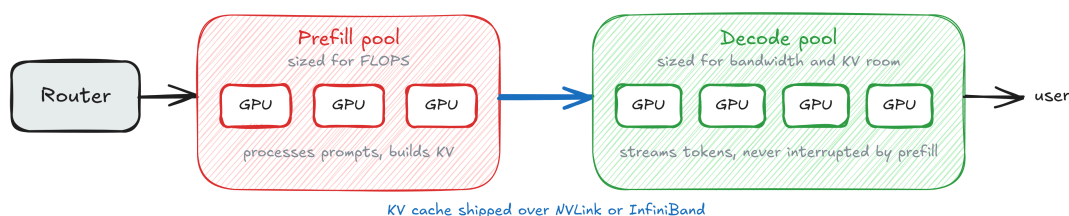
Prefix caching keeps those blocks after their request finishes and lets a new request reuse them if its prompt begins with exactly the same tokens. The engine then runs prefill only on the part after the shared prefix. For a 2,500-token prompt where 2,000 tokens are a shared system prompt, prefill drops to 500 tokens: five times less compute and five times less TTFT. SGLang's RadixAttention organizes the cache as a tree of prefixes so that many different continuations share as much as they have in common.

Two conditions have to hold, and they are what to ask about when someone claims a caching win. The prefix must match exactly from the first token, so prompts should be built with the stable part first and the variable part last; a timestamp at the top of the system prompt defeats the cache for every request. And the request has to land on a replica that holds the prefix. With ten replicas and random routing, the hit rate would be one in ten; cache-aware routing sends the request to the replica that already has it. The cache hit rate is the number that proves the cache is paying.

Disaggregated prefill and decode, step by step

On a single replica, prefill and decode compete for the same GPU. A long prefill occupies the arithmetic units for hundreds of milliseconds and every other request's decode step waits. Chunked prefill and prioritization soften this. Disaggregation removes it.

Disaggregated serving: prefill and decode on separate GPU pools



Each pool scales on its own. NVIDIA reported at the Dynamo launch (March 2025) that disaggregation more than doubled Llama 70B throughput on Hopper GPUs.

Two pools of GPUs are run: one that only does prefill, sized for arithmetic, and one that only does decode, sized for bandwidth and KV room. A request's prompt is processed in the prefill pool, its KV cache is shipped over NVLink or InfiniBand to a decode GPU, and the decode pool generates the answer without ever being interrupted by someone else's prefill. Each pool scales on its own. NVIDIA's Dynamo framework, announced in March 2025, provides the orchestration and the KV transfer layer. At launch NVIDIA reported that disaggregation more than doubled throughput for Llama 70B on Hopper GPUs and served up to 30 times more requests for DeepSeek-R1 on a GB200 NVL72 system; the gains depend heavily on model size and workload shape.

The cost is real: two pools to run, a KV transfer on every request, and an orchestration layer. It pays at scale and for workloads whose inputs and outputs are both long. For a small deployment the overhead exceeds the benefit, which is why it is a later lever, not a first one.

The engines, briefly

vLLM is open source, written in Python, and the reference implementation of PagedAttention and continuous batching. It has the broadest model support, adopts new architectures fastest, and is the default when flexibility matters.

SGLang is open source with a similar design, plus RadixAttention for tree-structured prefix caching and a fast structured-output path. It is often ahead on multi-turn and agentic workloads where prefixes repeat heavily.

TensorRT-LLM is NVIDIA's engine. It compiles the model into optimized kernels for a specific GPU and configuration ahead of time, and usually has the highest throughput and lowest latency on NVIDIA hardware, at the cost of build time and less flexibility.

NVIDIA Dynamo is not an engine but a distributed serving layer above them: disaggregation, KV-aware routing, KV transfer, multi-node coordination.

Managed inference platforms typically benchmark each model across these and pick the best per model, then add their own routing, caching, and autoscaling on top. That layer is the subject of Part 3.

Check yourself

1. Why does speculative decoding help at low batch and hurt at high batch? At low batch the GPU has spare arithmetic, so verifying five draft tokens in one target step costs about the same as one normal step; accepted tokens are effectively free. At high batch the GPU is compute-bound, verification costs five tokens' worth of arithmetic, and the draft consumes GPU time too.

2. What does prefix caching actually skip? The prefill compute for the shared prefix. The KV blocks already exist from an earlier request, so the engine runs prefill only on the new suffix. It pays only when prefixes repeat exactly from the first token and when the request lands on a replica that holds them.

3. A deployment's preemption counter is climbing. What does that tell you? The KV cache is running out under the context lengths being served. The concurrency setting is too high for the workload, or the workload's prompts have grown. Lower concurrency, cap the sequence length, quantize or offload the cache, or add memory.

4. When is disaggregated serving not worth it? For small deployments, where running two pools and shipping KV cache on every request costs more than the interference it removes. It pays at scale with long inputs and long outputs.

Part 3 covers metrics and operations: what every number on a benchmark table is physically made of, how to rebuild the throughput and latency curves from Part 1B, why queueing makes p99 explode while p50 barely moves, and how an autoscaler is really a statement about cold starts.

Radx Radhakrishnan works on LLM inference and agentic systems, after a decade of performance work on enterprise data platforms.

Sources

- Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI 2022 (iteration-level scheduling, the basis of continuous batching). <https://www.usenix.org/conference/osdi22/presentation/yu>
- Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP 2023 (near-zero KV cache waste; 2 to 4 × throughput at equal latency). <https://arxiv.org/abs/2309.06180>
- vLLM source, cache configuration (default block size 16 tokens). <https://github.com/vllm-project/vllm/blob/main/vllm/config/cache.py>
- Zheng et al., "SGLang: Efficient Execution of Structured Language Model Programs," 2023/2024 (RadixAttention). <https://arxiv.org/abs/2312.07104>

- Leviathan et al., "Fast Inference from Transformers via Speculative Decoding," ICML 2023 (2 to 3× speedup, identical output distribution). <https://arxiv.org/abs/2211.17192>
- Chen et al., "Accelerating Large Language Model Decoding with Speculative Sampling," 2023. <https://arxiv.org/abs/2302.01318>
- Cai et al., "Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads," 2024. <https://arxiv.org/abs/2401.10774>
- Li et al., "EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty," 2024. <https://arxiv.org/abs/2401.15077>
- Fu et al., "Break the Sequential Dependency of LLM Inference Using Lookahead Decoding," 2024. <https://arxiv.org/abs/2402.02057>
- NVIDIA, "Introducing NVIDIA Dynamo," March 18, 2025 (disaggregated serving results). <https://developer.nvidia.com/blog/introducing-nvidia-dynamo-a-low-latency-distributed-inference-framework-for-scaling-reasoning-ai-models/>
- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024 (671B total, 37B activated). <https://arxiv.org/abs/2412.19437>